

Building a CNN for MNIST Handwritten Digit Classification

Introduction

Welcome! In this assignment, you will build a Convolutional Neural Network (CNN) to classify handwritten digits from the famous MNIST dataset. This dataset is a classic in the field of computer vision and provides a great starting point for understanding image classification with deep learning.

This notebook is structured to guide you step-by-step through the process. You will load the data, preprocess it, define a CNN model, train it, and evaluate its performance. Throughout the assignment, you will have opportunities to experiment and deepen your understanding of the concepts.

Remember to:

- **Read all instructions carefully.**
- **Execute the code cells in order.**
- **Fill in the missing code sections marked as "Students: Fill in the blanks".**
- **Answer the reflection questions in the designated Markdown cell.**
- **Experiment and explore!** Change parameters, layers, and observe the effects.

Let's get started and build our MNIST digit classifier!

Section 1: Setting Up - Imports

Before we dive into building our CNN, we need to import the necessary libraries. These libraries provide pre-built tools and functions that will make our work much easier.

Instructions:

1. **Carefully review the code cell below.** It imports libraries from TensorFlow and Keras, which are powerful frameworks for building and training neural networks.
2. **Execute the code cell by selecting it and pressing [Shift + Enter] (or the "Run" button).**
3. **Ensure there are no error messages after running the cell.** If you encounter errors, double-check that you have TensorFlow and Keras installed in your environment.

```
# Cell 1: Imports
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
```

Explanation of Imports:

- **tensorflow as tf and keras:** TensorFlow is the main deep learning framework, and Keras is its high-level API that simplifies building and training models. We import TensorFlow as `tf` and Keras directly for easy access to their functionalities.
- **from tensorflow.keras import layers:** This imports the `layers` module from Keras, which provides various layers for building neural networks (like convolutional layers, dense layers, etc.).
- **from tensorflow.keras.datasets import mnist:** This imports the MNIST dataset directly from Keras datasets. This is very convenient for loading and using the MNIST data.
- **from tensorflow.keras.utils import to_categorical:** This imports the `to_categorical` function, which we will use to perform one-hot encoding of our labels.

Section 2: Data Loading and Preprocessing

In this section, we will load the MNIST dataset and prepare it for training our CNN model. Preprocessing steps are crucial to ensure our data is in the right format for the model to learn effectively.

Instructions:

1. **Read through the code in the cell below.** Understand how it loads the MNIST dataset and what preprocessing steps are applied.
2. **Execute the code cell.**
3. **Examine the comments in the code** to understand each preprocessing step in detail.

```
# Cell 2: Data Loading and Preprocessing
# Load the MNIST dataset (60k training images, 10k test images)
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to be between 0 and 1 (pixel values range from 0-255. scale to 0-1 by dividing by 255)
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add a channel dimension (for grayscale images, it's 1)
x_train = x_train.reshape(-1, 28, 28, 1) # (batch_size, height, width, channels) reshaping 28 x 28 to 28x 28 x 1, the -1 mean figure out batch size automatically
x_test = x_test.reshape(-1, 28, 28, 1)

# One-hot encode the labels
num_classes = 10 # 10 classes - digits 0-9
y_train = to_categorical(y_train, num_classes) # turn into one-hot format
y_test = to_categorical(y_test, num_classes)
```

Explanation of Data Preprocessing:

- **Loading the MNIST dataset:** `mnist.load_data()` loads the MNIST dataset, which is already split into training and testing sets (`(x_train, y_train), (x_test, y_test)`). `x_train` and `x_test` contain the images (pixel data), and `y_train` and `y_test` contain the corresponding labels (digits 0-9).
- **Normalization:** `x_train = x_train.astype("float32") / 255.0` and `x_test = x_test.astype("float32") / 255.0` normalize the pixel values. Pixel values in images are typically in the range 0-255. Dividing by 255 scales them to the range 0-1. This normalization helps the neural network train faster and more effectively.

- **Adding Channel Dimension:** `x_train = x_train.reshape(-1, 28, 28, 1)` and `x_test = x_test.reshape(-1, 28, 28, 1)` reshape the data to add a channel dimension. Even though MNIST images are grayscale (single channel), CNNs in Keras expect input data to have a channel dimension. We reshape from `(number_of_images, 28, 28)` to `(number_of_images, 28, 28, 1)`. The `-1` in `reshape` means "infer the dimension based on the size of the array."
- **One-Hot Encoding:** `y_train = to_categorical(y_train, num_classes)` and `y_test = to_categorical(y_test, num_classes)` perform one-hot encoding on the labels. Instead of representing the digit '3' as a single number, one-hot encoding converts it into a vector `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`, where the 4th position (index 3) is 'hot' (value 1), and all other positions are 'cold' (value 0). This is a standard way to represent categorical labels for neural networks in multi-class classification problems. `num_classes = 10` specifies that we have 10 classes (digits 0-9).

Section 3: Model Definition - Building the CNN

Now we will define the architecture of our Convolutional Neural Network (CNN). You will be building a sequential model using Keras layers.

Instructions:

1. **Carefully examine the code in the cell below.** Notice the structure of the `keras.Sequential` model.
2. **Fill in the missing parts** marked with `# Students: Fill in the blanks` to complete the model definition.
3. **Experiment!** You are encouraged to try different configurations for the layers, such as changing the number of filters in the convolutional layers, or adding more layers.

```
# Cell 3: Model Definition
# Build the CNN model. Students: Fill in the missing parts!
model = keras.Sequential(
    [
        keras.Input(shape=(28, 28, 1)), # Input layer - each image is 28 x 28 pixels and 1 channel

        layers.Conv2D(32, kernel_size=(3, 3), activation="relu"), # Convolutional layer 1 - uses 32 filters, each is 3 x3 pixels, ReLU to keep only positive values
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 1 Shrink each image by taking strongest pixel in each 2 x 2 area to make model faster, prevent overfitting, keep imp

        # Students: Add another Conv2D layer here. Experiment with the number of filters!
        # layers.Conv2D(128, kernel_size=(3, 3), activation="relu"), # Convolutional layer 2 - more filters more complex shape
        layers.Conv2D(128, kernel_size=(3, 3), activation="relu"), # Convolutional layer 2
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 2 - Shrink each image and keep important features

        # Students: Add another MaxPooling2D layer here if needed.
        layers.Conv2D(128, kernel_size=(3, 3), activation="relu"), # Convolutional layer 3
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 3
        layers.Flatten(), # Flatten layer turns 2D into 1D vector
        layers.Dropout(0.5), # Dropout layer - randomly turn off half of the neurons during training to prevent overfitting and learn general patterns
        layers.Dense(num_classes, activation="softmax"), # Output layer - 10 classes - 1 for each digit 0-9, softmax turns output into probabilities
    ]
)
```

Explanation of Layers:

- `keras.Input(shape=(28, 28, 1))`: This is the input layer of our model. It specifies the shape of the input images, which are 28x28 pixels with 1 channel (grayscale).
- `layers.Conv2D(32, kernel_size=(3, 3), activation="relu")`: This is a 2D Convolutional layer.
 - `32`: This is the number of filters (also called kernels). Each filter learns to detect specific features in the input image.
 - `kernel_size=(3, 3)`: This defines the size of the convolutional filter as 3x3 pixels.
 - `activation="relu"`: ReLU (Rectified Linear Unit) is the activation function. It introduces non-linearity into the model, allowing it to learn complex patterns.
- `layers.MaxPooling2D(pool_size=(2, 2))`: This is a Max Pooling layer.
 - `pool_size=(2, 2)`: It reduces the spatial dimensions of the feature maps by taking the maximum value within each 2x2 window. This helps to reduce the number of parameters, control overfitting, and make the model more robust to small shifts and distortions in the input.
- `layers.Flatten()`: This layer flattens the 2D feature maps from the convolutional and pooling layers into a 1D vector. This is necessary to connect the convolutional part of the network to the fully connected (Dense) layers.
- `layers.Dropout(0.5)`: This is a Dropout layer.
 - `0.5`: This sets the dropout rate to 50%. During training, this layer randomly sets 50% of the input units to 0 at each update. This is a regularization technique that helps to prevent overfitting.
- `layers.Dense(num_classes, activation="softmax")`: This is the output Dense (fully connected) layer.
 - `num_classes`: This is set to 10 because we have 10 classes (digits 0-9).
 - `activation="softmax"`: Softmax activation ensures that the output values are probabilities, and they sum up to 1 across all classes. The output will be a vector of 10 probabilities, where each probability represents the model's confidence that the input image belongs to that specific digit class.

Section 4: Model Compilation - Choosing Loss and Optimizer

Before we can train our model, we need to compile it. Compilation involves choosing an optimizer, a loss function, and metrics to evaluate the model's performance.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for the `loss` and `optimizer` parameters in `model.compile()`.
2. **Choose an appropriate loss function and optimizer** for this multi-class classification problem.
3. **In the Markdown cell after the code, explain your choices.** Why are these choices suitable for this task?

```
# Cell 4: Model Compilation
# Students: Choose an appropriate loss function and optimizer. Why did you choose these?
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"]) #Students: Fill in the blanks - cross entropy = how wrong, adam - adjust learning rates automati
# noisy gradients, works great for image tasks, stable for beginners and experts
```

Explanation of Choices (To be filled by students in the reflection section):

- **Loss Function:** You need to choose a loss function that is appropriate for multi-class classification. Think about what kind of error we are trying to minimize when classifying digits into 10 categories.
- **Optimizer:** You need to choose an optimizer that will efficiently update the model's weights to minimize the loss function. Consider common optimizers used in deep learning.
- **Metrics:** We are using "accuracy" as a metric to evaluate the model's performance. Accuracy is a common metric for classification tasks, representing the percentage of correctly classified images.

Section 5: Model Training - Fitting the Model to the Data

Now it's time to train our CNN model using the training data. Training involves feeding the training data to the model and adjusting its weights to minimize the loss function.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for `batch_size` and `epochs` in `model.fit()`.
2. **Choose appropriate values for `batch_size` and `epochs`.**
3. **Run the code cell to start training.** Observe the training progress, especially the loss and accuracy on both the training and validation sets.
4. **Experiment!** Change the `batch_size` and `epochs` and see how it affects the training process and the final performance.

```
# Cell 5: Model Training
# Students: Adjust the batch size and number of epochs. What happens if you change them?
model.fit(x_train, y_train, batch_size=512, epochs=25, validation_split=0.1) #Students: Fill in the blanks

Epoch 1/25
106/106 ————— 2s 17ms/step - accuracy: 0.9987 - loss: 0.0037 - val_accuracy: 0.9922 - val_loss: 0.0612
Epoch 2/25
106/106 ————— 2s 15ms/step - accuracy: 0.9989 - loss: 0.0036 - val_accuracy: 0.9925 - val_loss: 0.0582
Epoch 3/25
106/106 ————— 2s 14ms/step - accuracy: 0.9993 - loss: 0.0026 - val_accuracy: 0.9922 - val_loss: 0.0619
Epoch 4/25
106/106 ————— 2s 15ms/step - accuracy: 0.9994 - loss: 0.0023 - val_accuracy: 0.9930 - val_loss: 0.0599
Epoch 5/25
106/106 ————— 2s 14ms/step - accuracy: 0.9994 - loss: 0.0024 - val_accuracy: 0.9922 - val_loss: 0.0609
Epoch 6/25
106/106 ————— 2s 15ms/step - accuracy: 0.9991 - loss: 0.0021 - val_accuracy: 0.9927 - val_loss: 0.0593
Epoch 7/25
106/106 ————— 2s 14ms/step - accuracy: 0.9989 - loss: 0.0031 - val_accuracy: 0.9927 - val_loss: 0.0563
Epoch 8/25
106/106 ————— 2s 15ms/step - accuracy: 0.9991 - loss: 0.0029 - val_accuracy: 0.9928 - val_loss: 0.0572
Epoch 9/25
106/106 ————— 2s 16ms/step - accuracy: 0.9993 - loss: 0.0021 - val_accuracy: 0.9922 - val_loss: 0.0594
Epoch 10/25
106/106 ————— 2s 14ms/step - accuracy: 0.9990 - loss: 0.0030 - val_accuracy: 0.9935 - val_loss: 0.0577
Epoch 11/25
106/106 ————— 2s 15ms/step - accuracy: 0.9992 - loss: 0.0026 - val_accuracy: 0.9927 - val_loss: 0.0598
Epoch 12/25
106/106 ————— 2s 14ms/step - accuracy: 0.9991 - loss: 0.0026 - val_accuracy: 0.9928 - val_loss: 0.0570
Epoch 13/25
106/106 ————— 2s 14ms/step - accuracy: 0.9991 - loss: 0.0026 - val_accuracy: 0.9930 - val_loss: 0.0558
Epoch 14/25
106/106 ————— 2s 14ms/step - accuracy: 0.9990 - loss: 0.0028 - val_accuracy: 0.9915 - val_loss: 0.0600
Epoch 15/25
106/106 ————— 2s 14ms/step - accuracy: 0.9990 - loss: 0.0026 - val_accuracy: 0.9920 - val_loss: 0.0585
Epoch 16/25
106/106 ————— 2s 15ms/step - accuracy: 0.9992 - loss: 0.0026 - val_accuracy: 0.9915 - val_loss: 0.0618
Epoch 17/25
106/106 ————— 2s 16ms/step - accuracy: 0.9993 - loss: 0.0023 - val_accuracy: 0.9933 - val_loss: 0.0579
Epoch 18/25
106/106 ————— 2s 14ms/step - accuracy: 0.9993 - loss: 0.0021 - val_accuracy: 0.9920 - val_loss: 0.0633
Epoch 19/25
106/106 ————— 2s 14ms/step - accuracy: 0.9993 - loss: 0.0022 - val_accuracy: 0.9918 - val_loss: 0.0650
Epoch 20/25
106/106 ————— 2s 14ms/step - accuracy: 0.9989 - loss: 0.0030 - val_accuracy: 0.9927 - val_loss: 0.0645
Epoch 21/25
106/106 ————— 2s 14ms/step - accuracy: 0.9986 - loss: 0.0038 - val_accuracy: 0.9923 - val_loss: 0.0632
Epoch 22/25
106/106 ————— 2s 14ms/step - accuracy: 0.9989 - loss: 0.0033 - val_accuracy: 0.9920 - val_loss: 0.0630
Epoch 23/25
106/106 ————— 2s 14ms/step - accuracy: 0.9991 - loss: 0.0028 - val_accuracy: 0.9920 - val_loss: 0.0655
Epoch 24/25
106/106 ————— 2s 15ms/step - accuracy: 0.9990 - loss: 0.0025 - val_accuracy: 0.9918 - val_loss: 0.0659
Epoch 25/25
106/106 ————— 2s 15ms/step - accuracy: 0.9988 - loss: 0.0039 - val_accuracy: 0.9925 - val_loss: 0.0638
<keras.src.callbacks.history.History at 0x7b801b507130>
```

Explanation of Training Parameters:

- **`batch_size`**: This determines the number of training samples processed in each mini-batch during training. A larger batch size can speed up training but might require more memory. A smaller batch size can lead to more noisy updates but might generalize better.
- **`epochs`**: One epoch represents one complete pass through the entire training dataset. More epochs can potentially lead to better training but also increase the risk of overfitting, where the model learns the training data too well and performs poorly on unseen data.
- **`validation_split=0.1`**: This reserves 10% of the training data as a validation set. During training, the model's performance is evaluated on this validation set after each epoch. This helps to monitor for overfitting and tune hyperparameters.

Section 6: Model Evaluation - Assessing Performance on Test Data

After training, we need to evaluate our model's performance on the test dataset. This gives us an estimate of how well the model generalizes to unseen data.

Instructions:

1. **Run the code cell below.**
2. **Observe the output.** It will print the test loss and test accuracy.
3. **Think about the results.** Is the test accuracy satisfactory? How does it compare to the training and validation accuracy you observed during training?

```
# Cell 6: Model Evaluation
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
```

```
print(f"Test loss: {loss:.4f}")
print(f"Test accuracy: {accuracy:.4f}")

Test loss: 0.0559
Test accuracy: 0.9915
```

Section 7: Reflection and Answers to Questions

This is an important section! Take some time to reflect on what you have learned and answer the following questions in detail. Your thoughtful answers will demonstrate your understanding of the concepts covered in this assignment.

Reflection Questions:

1. **Conv2D Layer:** What is the role of the Conv2D layer? How do the `kernel_size` and the number of filters affect the learning process? *Hint: Experiment by changing these values in Cell 3.*

The Conv2D layer is responsible for learning features from the images. The kernel size affects how large the receptive field is and how much detail the filter can see. 3 x 3 is standard because it captures small patterns without losing detail and I chose not to experiment with changing the kernel size since I would go in knowing that the kernel size was suboptimal and didn't want to waste time and computer resources.

In my model, I increased the filters to 128. More filters means more feature extractions. It also meant that it needed more compute resources which wasn't much in this tiny learning model. By increasing the filters, it improved the model accuracy slightly but it also made it more sensitive to pooling and number of epochs.

2. **MaxPooling2D Layer:** What is the purpose of the MaxPooling2D layer? How does it contribute to the model's performance? *Hint: Try removing or adding a MaxPooling2D layer and see what happens.* To shrink the model and make it faster. I added the third maxpooling layer and it still worked but it became really sensitive because the MNIST images are small. If you pool too much then the images are shrunk down super aggressively.

I was able to get to about 99.1% accuracy but it also increased test loss.

3. **One-Hot Encoding:** Why do we use one-hot encoding for the labels?

categorical cross-entropy requires one-hot encoding. Since softmax uses percentages, the labels also need to be converted into vectors.

4. **Flatten Layer:** Why do we need the Flatten layer before the Dense layer?

To convert the 2D layer into a 1D vector so the dense layer can process them. The flatten layer is translating the convolutional layer's learned features so the Dense layer can make the final classification decision.

5. **Optimizer and Loss Function:** What optimizer and loss function did you choose in Cell 4? Explain your choices. Why is categorical cross-entropy a suitable loss function for this task? Why is Adam a good choice of optimiser?

I used:

Loss: categorical_crossentropy optimizer: Adam

Categorical cross-entropy is used when there are multiple class classifications and one-hot encoded labels. It will measure how "wrong" or how far from the predicted probability distribution is from the true distribution.

Adam adapts the learning rate automatically and learns quickly. It is also stable with deeper models. It seemed to give strong results across different batch sizes and epochs.

6. **Batch Size and Epochs:** How did you choose the batch size and number of epochs in Cell 5? What are the effects of changing these parameters? *Hint: Experiment!*

I just doubled my batch sizes from one run to the next. I increased the epochs from 15 to 25 and run them for three batch sizes that doubled the previous, 128, 256, 512, and then I compared all the statistics to see what did the best. I used Colab Tpu so it didn't take much time to experiment but it wasn't instant either.

At 15 epochs, the batch sizes all seemed to perform similarly. When I increased the epochs to 25, the larger batch sizes accuracy improved. But, the test loss also increased at 25 epochs.

The number of epochs affects how long the models learn and the batch size affects the gradient stability. It also affects generalization and noise. My experiments showed that the more epochs run would help the deeper model, more so when the batches were larger as well.

7. **Dropout:** Why is the Dropout layer included in the model?

To prevent overfitting and forces the model to learn the general patterns instead of memorizing them.

8. **Model Architecture:** Describe the overall architecture of your CNN. How many convolutional layers did you use? How many max pooling layers? What is the final dense layer doing?

I used: 3 Conv2D layers 3 maxpooling layers. 1 Flatten layer 1 Dense layer 1 Dense softmax output layer

The final dense layer is producing a probability that the image belongs to that image class.

9. **Performance:** What accuracy did you achieve on the test set? Are you happy with the result? Why or why not? If you're not happy, what could you try to improve the performance?

My best test accuracy was: .9915

I am happy with the result because it was much better than the baseline CNN and also showed the effects of batch size and number of epochs run. I could get better performance if I removed the third pooling layer. The model was kept small to get quick results and compare them. I didn't want to spend all day waiting for output.

Tips and Explanations:

- **Normalization:** Dividing the pixel values by 255 normalizes them to the range [0, 1]. This is important for training neural networks.
- **Reshaping:** The `reshape` operation adds a channel dimension to the images. For grayscale images, the channel dimension is 1.
- **One-Hot Encoding:** `to_categorical` converts the class labels (0-9) into one-hot encoded vectors.
- **Conv2D Parameters:** The `kernel_size` determines the size of the convolutional filter (e.g., 3x3). The number of filters determines how many different features are learned.
- **MaxPooling2D Parameters:** The `pool_size` determines the size of the pooling window (e.g., 2x2).
- **Optimizer:** The optimizer is the algorithm used to update the model's weights during training.
- **Loss Function:** The loss function measures the error between the model's predictions and the true labels.
- **Batch Size:** The batch size is the number of samples processed in each training iteration.

- **Epochs:** An epoch is one complete pass through the entire training dataset.
- **Dropout:** Dropout is a regularization technique that helps prevent overfitting.

Remember to run each cell to see its output. Experiment with the code and try to understand how different parameters affect the model's performance. Good luck! """"

Conclusion and Submission

Congratulations on completing this notebook assignment! You have successfully built and trained a Convolutional Neural Network to classify handwritten digits from the MNIST dataset. You've explored key concepts like convolutional layers, pooling layers, activation functions, optimizers, loss functions, and training procedures. To further solidify your understanding, consider the following:

- **Review your notebook:** Go back through each section, reread the explanations, and make sure you understand the code and the concepts.
- **Experiment further:** Try different CNN architectures, add more layers, change hyperparameters, and see how it affects the performance. Explore other optimizers or loss functions.
- **Reflect on your learning:** Think about the challenges you faced and how you overcame them. What were the most important takeaways for you from this assignment?

Submission Instructions

To submit your assignment:

1. **Save your notebook:** Ensure all your work, including code cells, outputs, and answers to reflection questions, is saved in the notebook.
2. **Print the notebook as a .pdf file** and submit it to Canvas.

Deadline: February, 12th

Building a CNN for MNIST Handwritten Digit Classification

✓ Introduction

Welcome! In this assignment, you will build a Convolutional Neural Network (CNN) to classify handwritten digits from the famous MNIST dataset. This dataset is a classic in the field of computer vision and provides a great starting point for understanding image classification with deep learning.

This notebook is structured to guide you step-by-step through the process. You will load the data, preprocess it, define a CNN model, train it, and evaluate its performance. Throughout the assignment, you will have opportunities to experiment and deepen your understanding of the concepts.

Remember to:

- **Read all instructions carefully.**
- **Execute the code cells in order.**
- **Fill in the missing code sections marked as "Students: Fill in the blanks".**
- **Answer the reflection questions in the designated Markdown cell.**
- **Experiment and explore!** Change parameters, layers, and observe the effects.

Let's get started and build our MNIST digit classifier!

Section 1: Setting Up - Imports

Before we dive into building our CNN, we need to import the necessary libraries. These libraries provide pre-built tools and functions that will make our work much easier.

Instructions:

1. **Carefully review the code cell below.** It imports libraries from TensorFlow and Keras, which are powerful frameworks for building and training neural networks.
2. **Execute the code cell by selecting it and pressing [Shift + Enter] (or the "Run" button).**
3. **Ensure there are no error messages after running the cell.** If you encounter errors, double-check that you have TensorFlow and Keras installed in your environment.

```
# Cell 1: Imports
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
```

Explanation of Imports:

- **tensorflow as tf and keras**: TensorFlow is the main deep learning framework, and Keras is its high-level API that simplifies building and training models. We import TensorFlow as `tf` and Keras directly for easy access to their functionalities.
- **from tensorflow.keras import layers**: This imports the `layers` module from Keras, which provides various layers for building neural networks (like convolutional layers, dense layers, etc.).
- **from tensorflow.keras.datasets import mnist**: This imports the MNIST dataset directly from Keras datasets. This is very convenient for loading and using the MNIST data.
- **from tensorflow.keras.utils import to_categorical**: This imports the `to_categorical` function, which we will use to perform one-hot encoding of our labels.

✓ Section 2: Data Loading and Preprocessing

In this section, we will load the MNIST dataset and prepare it for training our CNN model. Preprocessing steps are crucial to ensure our data is in the right format for the model to learn effectively.

Instructions:

1. **Read through the code in the cell below.** Understand how it loads the MNIST dataset and what preprocessing steps are applied.
2. **Execute the code cell.**
3. **Examine the comments in the code** to understand each preprocessing step in detail.

```
# Cell 2: Data Loading and Preprocessing
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add a channel dimension (for grayscale images, it's 1)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# One-hot encode the labels
num_classes = 10
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
```

Explanation of Data Preprocessing:

- **Loading the MNIST dataset:** `mnist.load_data()` loads the MNIST dataset, which is already split into training and testing sets `((x_train, y_train), (x_test, y_test))`. `x_train` and `x_test` contain the images (pixel data), and `y_train` and `y_test` contain the corresponding labels (digits 0-9).
- **Normalization:** `x_train = x_train.astype("float32") / 255.0` and `x_test = x_test.astype("float32") / 255.0` normalize the pixel values. Pixel values in images are typically in the range 0-255. Dividing by 255 scales them to the range 0-1. This normalization helps the neural network train faster and more effectively.
- **Adding Channel Dimension:** `x_train = x_train.reshape(-1, 28, 28, 1)` and `x_test = x_test.reshape(-1, 28, 28, 1)` reshape the data to add a channel dimension. Even though MNIST images are grayscale (single channel), CNNs in Keras expect input data to have a channel dimension. We reshape from `(number_of_images, 28, 28)` to `(number_of_images, 28, 28, 1)`. The `-1` in `reshape` means "infer the dimension based on the size of the array."
- **One-Hot Encoding:** `y_train = to_categorical(y_train, num_classes)` and `y_test = to_categorical(y_test, num_classes)` perform one-hot encoding on the labels. Instead of representing the digit '3' as a single number, one-hot encoding converts it into a vector `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`, where the 4th position (index 3) is 'hot' (value 1), and all other positions are 'cold' (value 0). This is a standard way to represent categorical labels for neural networks in multi-class classification problems. `num_classes = 10` specifies that we have 10 classes (digits 0-9).

Section 3: Model Definition - Building the CNN

Now we will define the architecture of our Convolutional Neural Network (CNN). You will be building a sequential model using Keras layers.

Instructions:

1. **Carefully examine the code in the cell below.** Notice the structure of the `keras.Sequential` model.
2. **Fill in the missing parts** marked with `# Students: Fill in the blanks` to complete the model definition.
3. **Experiment!** You are encouraged to try different configurations for the layers, such as changing the number of filters in the convolutional layers, or adding more layers.

```
# Cell 3: Model Definition
# Build the CNN model. Students: Fill in the missing parts!
model = keras.Sequential(
    [
        keras.Input(shape=(28, 28, 1)), # Input layer
        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"), # Convolutional layer 1
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 1
        # Students: Add another Conv2D layer here. Experiment with the number of filters!
        layers.Conv2D(24, kernel_size=(2, 2), activation="relu"), # Convolutional layer 2
        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"), # Convolutional layer 2
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 2
        # Students: Add another MaxPooling2D layer here if needed.
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 2
        layers.Flatten(), # Flatten layer
        layers.Dropout(0.5), # Dropout layer
        layers.Dense(num_classes, activation="softmax"), # Output layer
    ]
)
```

Explanation of Layers:

- `keras.Input(shape=(28, 28, 1))`: This is the input layer of our model. It specifies the shape of the input images, which are 28x28 pixels with 1 channel (grayscale).
- `layers.Conv2D(32, kernel_size=(3, 3), activation="relu")`: This is a 2D Convolutional layer.
 - `32`: This is the number of filters (also called kernels). Each filter learns to detect specific features in the input image.
 - `kernel_size=(3, 3)`: This defines the size of the convolutional filter as 3x3 pixels.
 - `activation="relu"`: ReLU (Rectified Linear Unit) is the activation function. It introduces non-linearity into the model, allowing it to learn complex patterns.
- `layers.MaxPooling2D(pool_size=(2, 2))`: This is a Max Pooling layer.
 - `pool_size=(2, 2)`: It reduces the spatial dimensions of the feature maps by taking the maximum value within each 2x2 window. This helps to reduce the number of parameters, control overfitting, and make the model more robust to small shifts and distortions in the input.
- `layers.Flatten()`: This layer flattens the 2D feature maps from the convolutional and pooling layers into a 1D vector. This is necessary to connect the convolutional part of the network to the fully connected (Dense) layers.
- `layers.Dropout(0.5)`: This is a Dropout layer.
 - `0.5`: This sets the dropout rate to 50%. During training, this layer randomly sets 50% of the input units to 0 at each update. This is a regularization technique that helps to prevent overfitting.
- `layers.Dense(num_classes, activation="softmax")`: This is the output Dense (fully connected) layer.
 - `num_classes`: This is set to 10 because we have 10 classes (digits 0-9).
 - `activation="softmax"`: Softmax activation ensures that the output values are probabilities, and they sum up to 1 across all classes. The output will be a vector of 10 probabilities, where each probability represents the model's confidence that the input image belongs to that specific digit class.

Section 4: Model Compilation - Choosing Loss and Optimizer

Before we can train our model, we need to compile it. Compilation involves choosing an optimizer, a loss function, and metrics to evaluate the model's performance.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for the `loss` and `optimizer` parameters in `model.compile()`.
2. **Choose an appropriate loss function and optimizer** for this multi-class classification problem.
3. **In the Markdown cell after the code, explain your choices.** Why are these choices suitable for this task?

```
# Cell 4: Model Compilation
# Students: Choose an appropriate loss function and optimizer. Why did you choose these?
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"]) #Students: Fill in the blanks
```

Explanation of Choices (To be filled by students in the reflection section):

- **Loss Function:** You need to choose a loss function that is appropriate for multi-class classification. Think about what kind of error we are trying to minimize when classifying digits into 10 categories.
- **Optimizer:** You need to choose an optimizer that will efficiently update the model's weights to minimize the loss function. Consider common optimizers used in deep learning.
- **Metrics:** We are using "accuracy" as a metric to evaluate the model's performance. Accuracy is a common metric for classification tasks, representing the percentage of correctly classified images.

Section 5: Model Training - Fitting the Model to the Data

Now it's time to train our CNN model using the training data. Training involves feeding the training data to the model and adjusting its weights to minimize the loss function.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for `batch_size` and `epochs` in `model.fit()`.
2. **Choose appropriate values for `batch_size` and `epochs`.**
3. **Run the code cell to start training.** Observe the training progress, especially the loss and accuracy on both the training and validation sets.
4. **Experiment!** Change the `batch_size` and `epochs` and see how it affects the training process and the final performance.

```
# Cell 5: Model Training
# Students: Adjust the batch size and number of epochs. What happens if you change them?
model.fit(x_train, y_train, batch_size=256, epochs=100, validation_split=0.1) #Students: Fill in the blanks

211/211 ————— 2s 9ms/step - accuracy: 0.9887 - loss: 0.0342 - val_accuracy: 0.9937 - val_loss: 0.0298
Epoch 73/100
211/211 ————— 2s 9ms/step - accuracy: 0.9889 - loss: 0.0335 - val_accuracy: 0.9925 - val_loss: 0.0308
Epoch 74/100
211/211 ————— 2s 9ms/step - accuracy: 0.9886 - loss: 0.0346 - val_accuracy: 0.9930 - val_loss: 0.0290
Epoch 75/100
211/211 ————— 2s 10ms/step - accuracy: 0.9897 - loss: 0.0320 - val_accuracy: 0.9932 - val_loss: 0.0285
Epoch 76/100
211/211 ————— 2s 9ms/step - accuracy: 0.9886 - loss: 0.0350 - val_accuracy: 0.9932 - val_loss: 0.0289
Epoch 77/100
211/211 ————— 2s 9ms/step - accuracy: 0.9897 - loss: 0.0309 - val_accuracy: 0.9927 - val_loss: 0.0304
Epoch 78/100
211/211 ————— 2s 9ms/step - accuracy: 0.9901 - loss: 0.0293 - val_accuracy: 0.9927 - val_loss: 0.0310
Epoch 79/100
211/211 ————— 2s 9ms/step - accuracy: 0.9905 - loss: 0.0287 - val_accuracy: 0.9933 - val_loss: 0.0304
Epoch 80/100
211/211 ————— 2s 9ms/step - accuracy: 0.9895 - loss: 0.0308 - val_accuracy: 0.9927 - val_loss: 0.0319
Epoch 81/100
211/211 ————— 2s 10ms/step - accuracy: 0.9896 - loss: 0.0314 - val_accuracy: 0.9930 - val_loss: 0.0320
Epoch 82/100
211/211 ————— 2s 10ms/step - accuracy: 0.9894 - loss: 0.0329 - val_accuracy: 0.9932 - val_loss: 0.0297
Epoch 83/100
211/211 ————— 2s 9ms/step - accuracy: 0.9896 - loss: 0.0310 - val_accuracy: 0.9932 - val_loss: 0.0312
Epoch 84/100
211/211 ————— 2s 9ms/step - accuracy: 0.9904 - loss: 0.0300 - val_accuracy: 0.9928 - val_loss: 0.0308
Epoch 85/100
211/211 ————— 2s 9ms/step - accuracy: 0.9900 - loss: 0.0297 - val_accuracy: 0.9935 - val_loss: 0.0295
Epoch 86/100
211/211 ————— 2s 9ms/step - accuracy: 0.9896 - loss: 0.0316 - val_accuracy: 0.9930 - val_loss: 0.0318
Epoch 87/100
211/211 ————— 2s 9ms/step - accuracy: 0.9907 - loss: 0.0298 - val_accuracy: 0.9938 - val_loss: 0.0291
Epoch 88/100
211/211 ————— 2s 10ms/step - accuracy: 0.9902 - loss: 0.0294 - val_accuracy: 0.9930 - val_loss: 0.0311
Epoch 89/100
211/211 ————— 2s 9ms/step - accuracy: 0.9898 - loss: 0.0311 - val_accuracy: 0.9938 - val_loss: 0.0305
Epoch 90/100
211/211 ————— 2s 9ms/step - accuracy: 0.9908 - loss: 0.0287 - val_accuracy: 0.9923 - val_loss: 0.0327
Epoch 91/100
211/211 ————— 2s 9ms/step - accuracy: 0.9900 - loss: 0.0298 - val_accuracy: 0.9942 - val_loss: 0.0298
Epoch 92/100
211/211 ————— 2s 9ms/step - accuracy: 0.9901 - loss: 0.0285 - val_accuracy: 0.9933 - val_loss: 0.0314
Epoch 93/100
211/211 ————— 2s 9ms/step - accuracy: 0.9911 - loss: 0.0282 - val_accuracy: 0.9935 - val_loss: 0.0327
Epoch 94/100
211/211 ————— 2s 10ms/step - accuracy: 0.9905 - loss: 0.0288 - val_accuracy: 0.9938 - val_loss: 0.0301
Epoch 95/100
211/211 ————— 2s 10ms/step - accuracy: 0.9901 - loss: 0.0288 - val_accuracy: 0.9935 - val_loss: 0.0290
Epoch 96/100
211/211 ————— 2s 9ms/step - accuracy: 0.9908 - loss: 0.0275 - val_accuracy: 0.9918 - val_loss: 0.0336
Epoch 97/100
211/211 ————— 2s 9ms/step - accuracy: 0.9909 - loss: 0.0273 - val_accuracy: 0.9928 - val_loss: 0.0313
Epoch 98/100
211/211 ————— 2s 9ms/step - accuracy: 0.9904 - loss: 0.0292 - val_accuracy: 0.9942 - val_loss: 0.0297
Epoch 99/100
211/211 ————— 2s 9ms/step - accuracy: 0.9915 - loss: 0.0272 - val_accuracy: 0.9935 - val_loss: 0.0294
Epoch 100/100
211/211 ————— 2s 9ms/step - accuracy: 0.9904 - loss: 0.0273 - val_accuracy: 0.9928 - val_loss: 0.0345
<keras.src.callbacks.history.History at 0x7858b5ff7680>
```

Explanation of Training Parameters:

- **batch_size**: This determines the number of training samples processed in each mini-batch during training. A larger batch size can speed up training but might require more memory. A smaller batch size can lead to more noisy updates but might generalize better.
- **epochs**: One epoch represents one complete pass through the entire training dataset. More epochs can potentially lead to better training but also increase the risk of overfitting, where the model learns the training data too well and performs poorly on unseen data.
- **validation_split=0.1**: This reserves 10% of the training data as a validation set. During training, the model's performance is evaluated on this validation set after each epoch. This helps to monitor for overfitting and tune hyperparameters.

Section 6: Model Evaluation - Assessing Performance on Test Data

After training, we need to evaluate our model's performance on the test dataset. This gives us an estimate of how well the model generalizes to unseen data.

Instructions:

1. **Run the code cell below.**
2. **Observe the output.** It will print the test loss and test accuracy.
3. **Think about the results.** Is the test accuracy satisfactory? How does it compare to the training and validation accuracy you observed during training?

```
# Cell 6: Model Evaluation
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {loss:.4f}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Test loss: 0.0233
Test accuracy: 0.9931
```

⌕ B I <> ↺ 🖼️ “ ⋮ ⋮ — Ψ 😊 ☰ Close

Section 7: Reflection and Answers to Questions

This is an important section! Take some time to reflect on what you have learned and answer the following questions in detail. Your thoughtful answers will demonstrate your understanding of the concepts covered in this assignment.

Reflection Questions:

1. ****Conv2D Layer:**** What is the role of the Conv2D layer? How do the `kernel\_size` and the number of filters affect the learning process? **Hint: Experiment by changing these values in Cell 3.**

The conv2d layer learns important features from images, like edges and shapes. The kernel_size controls how much of the image the filter looks at, while the number of filters controls how many features the model can learn. In my tests, 16 filters with a 3x3 kernel gave the best accuracy of 99.16%.

2. ****MaxPooling2D Layer:**** What is the purpose of the MaxPooling2D layer? How does it contribute to the model's performance? **Hint: Try removing or adding a MaxPooling2D layer and see what happens.**

The MaxPooling2D layer reduces the size of the feature maps while keeping important information. In my test, removing the MaxPooling layers reduced the accuracy from 98.99% to 98.90% and increased the loss. This shows that MaxPooling helped my model perform a little better.

3. ****One-Hot Encoding:**** Why do we use one-hot encoding for the labels?

We use one-hot encoding to represent each digit label as a separate class. Like the digit 3 is represented as a vector with 1 in the position for 3 and 0 in the other positions, this helps the CNN classify the digits correctly.

4. ****Flatten Layer:**** Why do we need the Flatten layer before the Dense layer?

The Flatten layer changes the feature maps into one long vector. We need it before the Dense layer because the Dense layer needs the data in this one-dimensional format to make the final classification.

5. ****Optimizer and Loss Function:**** What optimizer and loss function did you choose in Cell 4? Explain your choices. Why is categorical cross-entropy a suitable loss function for this task? Why is Adam a good choice of optimizer?

I chose the Adam optimizer and categorical cross-entropy loss. Categorical cross-entropy is suitable because the MNIST dataset has 10 different digit classes, and the labels are one-hot encoded. I chose Adam because it adjusts the learning rate during training, which works well for training neural networks.

6. ****Batch Size and Epochs:**** How did you choose the batch size and number of epochs in Cell 5? What are the effects of changing these

Section 7: Reflection and Answers to Questions

This is an important section! Take some time to reflect on what you have learned and answer the following questions in detail. Your thoughtful answers will demonstrate your understanding of the concepts covered in this assignment.

Reflection Questions:

1. **Conv2D Layer:** What is the role of the Conv2D layer? How do the `kernel_size` and the number of filters affect the learning process? *Hint: Experiment by changing these values in Cell 3.*

The conv2d layer learns important features from images, like edges and shapes. The kernel_size controls how much of the image the filter looks at, while the number of filters controls how many features the model can learn. In my tests, 16 filters with a 3x3 kernel gave the best accuracy of 99.16%.

2. **MaxPooling2D Layer:** What is the purpose of the MaxPooling2D layer? How does it contribute to the model's performance? *Hint: Try removing or adding a MaxPooling2D layer and see what happens.*

The MaxPooling2D layer reduces the size of the feature maps while keeping important information. In my test, removing the MaxPooling layers reduced the accuracy from 98.99% to 98.90% and increased the loss. This shows that MaxPooling helped my model perform a little better.

3. **One-Hot Encoding:** Why do we use one-hot encoding for the labels?

We use one-hot encoding to represent each digit label as a separate class. Like the digit 3 is represented as a vector with 1 in the position for 3 and 0 in the other positions, this helps the CNN classify the digits correctly.

4. **Flatten Layer:** Why do we need the Flatten layer before the Dense layer?

The Flatten layer changes the feature maps into one long vector. We need it before the Dense layer because the Dense layer needs the data in this one-dimensional format to make the final classification.

5. **Optimizer and Loss Function:** What optimizer and loss function did you choose in Cell 4? Explain your choices. Why is categorical cross-entropy a suitable loss function for this task? Why is Adam a good choice of optimizer?

parameters? **Hint: Experiment!**

The original model used a batch size of 128 and 15 epochs. After testing different values, a batch size of 256 and more epochs gave better results. With 100 epochs, the model reached 99.31% accuracy. Increasing the epochs improved the accuracy in the tests, but the improvement became smaller as the number of epochs increased.

7. ****Dropout:**** Why is the Dropout layer included in the model?

The Dropout layer is included to help prevent overfitting, it randomly turns off some neurons during training, so the model does not depend too much on specific neurons and can learn better patterns.

8. ****Model Architecture:**** Describe the overall architecture of your CNN. How many convolutional layers did you use? How many max pooling layers? What is the final dense layer doing?

My CNN has 3 convolutional layers and 3 maxpooling2d layers. After these layers, the feature maps go through a flatten layer and then dropout layer, the final dense layer has 10 outputs and is used to classify the image into one of the 10 digits from 0 to 9.

9. ****Performance:**** What accuracy did you achieve on the test set? Are you happy with the result? Why or why not? If you're not happy, what could you try to improve the performance?

I achieved 99.31% accuracy on the test set. I am happy with this result because the model was able to correctly classify almost all the handwritten digits.

****Tips and Explanations:****

* ****Normalization:**** Dividing the pixel values by 255 normalizes them to the range [0, 1]. This is important for training neural networks.

* ****Reshaping:**** The `reshape` operation adds a channel dimension to the images. For grayscale images, the channel dimension is 1.

* ****One-Hot Encoding:**** `to\_categorical` converts the class labels (0-9) into one-hot encoded vectors.

* ****Conv2D Parameters:**** The `kernel\_size` determines the size of the convolutional filter (e.g., 3x3). The number of filters determines how many different features are learned.

* ****MaxPooling2D Parameters:**** The `pool\_size` determines the size of the pooling window (e.g., 2x2).

* ****Optimizer:**** The optimizer is the algorithm used to update the model's weights during training.

* ****Loss Function:**** The loss function measures the error between the model's predictions and the true labels.

* ****Batch Size:**** The batch size is the number of samples processed in each training iteration.

* ****Epochs:**** An epoch is one complete pass through the entire training dataset.

* ****Dropout:**** Dropout is a regularization technique that helps prevent overfitting.

Remember to run each cell to see its output. Experiment with the code and try to understand how different parameters affect the model's performance. Good luck!

"""

I chose the Adam optimizer and categorical cross-entropy loss. Categorical cross-entropy is suitable because the MNIST dataset has 10 different digit classes, and the labels are one-hot encoded. I chose Adam because it adjusts the learning rate during training and works well for training neural networks.

6. **Batch Size and Epochs:** How did you choose the batch size and number of epochs in Cell 5? What are the effects of changing these parameters? *Hint: Experiment!*

The original model used a batch size of 128 and 15 epochs. After testing different values, a batch size of 256 and more epochs gave better results. With 100 epochs, the model reached 99.31% accuracy. Increasing the epochs improved the accuracy in the tests, but the improvement became smaller as the number of epochs increased.

7. **Dropout:** Why is the Dropout layer included in the model?

The Dropout layer is included to help prevent overfitting, it randomly turns off some neurons during training, so the model does not depend too much on specific neurons and can learn better patterns.

8. **Model Architecture:** Describe the overall architecture of your CNN. How many convolutional layers did you use? How many max pooling layers? What is the final dense layer doing?

My CNN has 3 convolutional layers and 3 maxpooling2d layers. After these layers, the feature maps go through a flatten layer and then a dropout layer, the final dense layer has 10 outputs and is used to classify the image into one of the 10 digits from 0 to 9.

9. **Performance:** What accuracy did you achieve on the test set? Are you happy with the result? Why or why not? If you're not happy, what could you try to improve the performance?

I achieved 99.31% accuracy on the test set. I am happy with this result because the model was able to correctly classify almost all of the handwritten digits.

Tips and Explanations:

- **Normalization:** Dividing the pixel values by 255 normalizes them to the range [0, 1]. This is important for training neural networks.

- **Reshaping:** The `reshape` operation adds a channel dimension to the images. For grayscale images, the channel dimension is 1.

- **One-Hot Encoding:** `to_categorical` converts the class labels (0-9) into one-hot encoded vectors.

- **Conv2D Parameters:** The `kernel_size` determines the size of the convolutional filter (e.g., 3x3). The number of filters determines how many different features are learned.

- **MaxPooling2D Parameters:** The `pool_size` determines the size of the pooling window (e.g., 2x2).

- **Optimizer:** The optimizer is the algorithm used to update the model's weights during training.

- **Loss Function:** The loss function measures the error between the model's predictions and the true labels.

- **Batch Size:** The batch size is the number of samples processed in each training iteration.

- **Epochs:** An epoch is one complete pass through the entire training dataset.
- **Dropout:** Dropout is a regularization technique that helps prevent overfitting.

Remember to run each cell to see its output. Experiment with the code and try to understand how different parameters affect the model's performance. Good luck! """"

Conclusion and Submission

Congratulations on completing this notebook assignment! You have successfully built and trained a Convolutional Neural Network to classify handwritten digits from the MNIST dataset. You've explored key concepts like convolutional layers, pooling layers, activation functions, optimizers, loss functions, and training procedures. To further solidify your understanding, consider the following:

- **Review your notebook:** Go back through each section, reread the explanations, and make sure you understand the code and the concepts.
- **Experiment further:** Try different CNN architectures, add more layers, change hyperparameters, and see how it affects the performance. Explore other optimizers or loss functions.
- **Reflect on your learning:** Think about the challenges you faced and how you overcame them. What were the most important takeaways for you from this assignment?

Submission Instructions

To submit your assignment:

1. **Save your notebook:** Ensure all your work, including code cells, outputs, and answers to reflection questions, is saved in the notebook.
2. **Print the notebook as a .pdf file** and submit it to Canvas.

Deadline: February, 12th

Introduction

Welcome! In this assignment, you will build a Convolutional Neural Network (CNN) to classify handwritten digits from the famous MNIST dataset. This dataset is a classic in the field of computer vision and provides a great starting point for understanding image classification with deep learning.

This notebook is structured to guide you step-by-step through the process. You will load the data, preprocess it, define a CNN model, train it, and evaluate its performance. Throughout the assignment, you will have opportunities to experiment and deepen your understanding of the concepts.

Remember to:

- **Read all instructions carefully.**
- **Execute the code cells in order.**
- **Fill in the missing code sections marked as "Students: Fill in the blanks".**
- **Answer the reflection questions in the designated Markdown cell.**
- **Experiment and explore!** Change parameters, layers, and observe the effects.

Let's get started and build our MNIST digit classifier!

Section 1: Setting Up - Imports

Before we dive into building our CNN, we need to import the necessary libraries. These libraries provide pre-built tools and functions that will make our work much easier.

Instructions:

1. **Carefully review the code cell below.** It imports libraries from TensorFlow and Keras, which are powerful frameworks for building and training neural networks.
2. **Execute the code cell by selecting it and pressing [Shift + Enter] (or the "Run" button).**
3. **Ensure there are no error messages after running the cell.** If you encounter errors, double-check that you have TensorFlow and Keras installed in your environment.

```
1 # Cell 1: Imports
2 import tensorflow as tf
3 from tensorflow import keras
4 from tensorflow.keras import layers
5 from tensorflow.keras.datasets import mnist
6 from tensorflow.keras.utils import to_categorical
```

Explanation of Imports:

- **tensorflow as tf** and **keras**: TensorFlow is the main deep learning framework, and Keras is its high-level API that simplifies building and training models. We import TensorFlow as **tf** and Keras directly for easy access to their functionalities.
- **from tensorflow.keras import layers**: This imports the **layers** module from Keras, which provides various layers for building neural networks (like convolutional layers, dense layers, etc.).
- **from tensorflow.keras.datasets import mnist**: This imports the MNIST dataset directly from Keras datasets. This is very convenient for loading and using the MNIST data.
- **from tensorflow.keras.utils import to_categorical**: This imports the **to_categorical** function, which we will use to perform one-hot encoding of our labels.

Section 2: Data Loading and Preprocessing

In this section, we will load the MNIST dataset and prepare it for training our CNN model. Preprocessing steps are crucial to ensure our data is in the right format for the model to learn effectively.

Instructions:

1. **Read through the code in the cell below.** Understand how it loads the MNIST dataset and what preprocessing steps are applied.
2. **Execute the code cell.**
3. **Examine the comments in the code** to understand each preprocessing step in detail.

```
1 # Cell 2: Data Loading and Preprocessing
2 # Load the MNIST dataset
```

```

3 (x_train, y_train), (x_test, y_test) = mnist.load_data()
4
5 # Normalize pixel values to be between 0 and 1
6 x_train = x_train.astype("float32") / 255.0
7 x_test = x_test.astype("float32") / 255.0
8
9 # Add a channel dimension (for grayscale images, it's 1)
10 x_train = x_train.reshape(-1, 28, 28, 1)
11 x_test = x_test.reshape(-1, 28, 28, 1)
12
13 # One-hot encode the labels
14 num_classes = 10
15 y_train = to_categorical(y_train, num_classes)
16 y_test = to_categorical(y_test, num_classes)

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 — 0s 0us/step

Explanation of Data Preprocessing:

- **Loading the MNIST dataset:** `mnist.load_data()` loads the MNIST dataset, which is already split into training and testing sets `((x_train, y_train), (x_test, y_test))`. `x_train` and `x_test` contain the images (pixel data), and `y_train` and `y_test` contain the corresponding labels (digits 0-9).
- **Normalization:** `x_train = x_train.astype("float32") / 255.0` and `x_test = x_test.astype("float32") / 255.0` normalize the pixel values. Pixel values in images are typically in the range 0-255. Dividing by 255 scales them to the range 0-1. This normalization helps the neural network train faster and more effectively.
- **Adding Channel Dimension:** `x_train = x_train.reshape(-1, 28, 28, 1)` and `x_test = x_test.reshape(-1, 28, 28, 1)` reshape the data to add a channel dimension. Even though MNIST images are grayscale (single channel), CNNs in Keras expect input data to have a channel dimension. We reshape from `(number_of_images, 28, 28)` to `(number_of_images, 28, 28, 1)`. The `-1` in `reshape` means "infer the dimension based on the size of the array."
- **One-Hot Encoding:** `y_train = to_categorical(y_train, num_classes)` and `y_test = to_categorical(y_test, num_classes)` perform one-hot encoding on the labels. Instead of representing the digit '3' as a single number, one-hot encoding converts it into a vector `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]` where the 4th position (index 3) is 'hot' (value 1), and all other positions are 'cold' (value 0). This is a standard way to represent categorical labels for neural networks in multi-class classification problems. `num_classes = 10` specifies that we have 10 classes (digits 0-9).

Section 3: Model Definition - Building the CNN

Now we will define the architecture of our Convolutional Neural Network (CNN). You will be building a sequential model using Keras layers.

Instructions:

1. **Carefully examine the code in the cell below.** Notice the structure of the `keras.Sequential` model.
2. **Fill in the missing parts** marked with `# Students: Fill in the blanks` to complete the model definition.
3. **Experiment!** You are encouraged to try different configurations for the layers, such as changing the number of filters in the convolutional layers, or adding more layers.

```

1 # Cell 3: Model Definition
2 # Build the CNN model. Students: Fill in the missing parts!
3 model = keras.Sequential(
4     [
5         keras.Input(shape=(28, 28, 1)), # Input layer
6         layers.Conv2D(32, kernel_size=(3, 3), activation="relu"), # Convolutional layer 1
7         layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 1
8         # Students: Add another Conv2D layer here. Experiment with the number of filters!
9         layers.Conv2D(____, kernel_size=(____, ____), activation="____"), # Convolutional layer 2
10        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"), # Convolutional layer 2
11        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 2
12        # Students: Add another MaxPooling2D layer here if needed.
13        layers.MaxPooling2D(pool_size=(____, ____)), # Max pooling layer 2
14        layers.Flatten(), # Flatten layer
15        layers.Dropout(0.5), # Dropout layer
16        layers.Dense(num_classes, activation="softmax"), # Output layer
17    ]
18 )

```

Architecture change:

Increase the second Conv2D layer from 64 to 128 filters. I will compare its effect later during training and evaluation.

```

1 model = keras.Sequential(
2     [
3         keras.Input(shape=(28, 28, 1)),
4         layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
5         layers.MaxPooling2D(pool_size=(2, 2)),
6
7         layers.Conv2D(128, kernel_size=(3, 3), activation="relu"),
8         layers.MaxPooling2D(pool_size=(2, 2)),
9
10        layers.Flatten(),
11        layers.Dropout(0.5),
12        layers.Dense(num_classes, activation="softmax"),
13    ]
14 )

```

Explanation of Layers:

- `keras.Input(shape=(28, 28, 1))`: This is the input layer of our model. It specifies the shape of the input images, which are 28x28 pixels with 1 channel (grayscale).
- `layers.Conv2D(32, kernel_size=(3, 3), activation="relu")`: This is a 2D Convolutional layer.
 - `32`: This is the number of filters (also called kernels). Each filter learns to detect specific features in the input image.
 - `kernel_size=(3, 3)`: This defines the size of the convolutional filter as 3x3 pixels.
 - `activation="relu"`: ReLU (Rectified Linear Unit) is the activation function. It introduces non-linearity into the model, allowing it to learn complex patterns.
- `layers.MaxPooling2D(pool_size=(2, 2))`: This is a Max Pooling layer.
 - `pool_size=(2, 2)`: It reduces the spatial dimensions of the feature maps by taking the maximum value within each 2x2 window. This helps to reduce the number of parameters, control overfitting, and make the model more robust to small shifts and distortions in the input.
- `layers.Flatten()`: This layer flattens the 2D feature maps from the convolutional and pooling layers into a 1D vector. This is necessary to connect the convolutional part of the network to the fully connected (Dense) layers.
- `layers.Dropout(0.5)`: This is a Dropout layer.
 - `0.5`: This sets the dropout rate to 50%. During training, this layer randomly sets 50% of the input units to 0 at each update. This is a regularization technique that helps to prevent overfitting.
- `layers.Dense(num_classes, activation="softmax")`: This is the output Dense (fully connected) layer.
 - `num_classes`: This is set to 10 because we have 10 classes (digits 0-9).
 - `activation="softmax"`: Softmax activation ensures that the output values are probabilities, and they sum up to 1 across all classes. The output will be a vector of 10 probabilities, where each probability represents the model's confidence that the input image belongs to that specific digit class.

Section 4: Model Compilation - Choosing Loss and Optimizer

Before we can train our model, we need to compile it. Compilation involves choosing an optimizer, a loss function, and metrics to evaluate the model's performance.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for the `loss` and `optimizer` parameters in `model.compile()`.
2. **Choose an appropriate loss function and optimizer** for this multi-class classification problem.
3. **In the Markdown cell after the code, explain your choices.** Why are these choices suitable for this task?

```

1 model.compile(
2     loss="categorical_crossentropy",
3     optimizer="adam",
4     metrics=["accuracy"]
5 )

```

Loss Function:

I chose categorical cross-entropy because this is a multi-class classification problem with 10 possible digit classes, and the labels were one-hot encoded during preprocessing. It measures the difference between the model's predicted probabilities and the correct class.

Optimizer:

I chose Adam because it efficiently updates the model’s weights during training and adapts the learning rate as training progresses.

Metric:

Accuracy is used because it shows the percentage of digit images the model classifies correctly.

Explanation of Choices (To be filled by students in the reflection section):

- **Loss Function:** You need to choose a loss function that is appropriate for multi-class classification. Think about what kind of error we are trying to minimize when classifying digits into 10 categories.
- **Optimizer:** You need to choose an optimizer that will efficiently update the model's weights to minimize the loss function. Consider common optimizers used in deep learning.
- **Metrics:** We are using "accuracy" as a metric to evaluate the model's performance. Accuracy is a common metric for classification tasks, representing the percentage of correctly classified images.

Section 5: Model Training - Fitting the Model to the Data

Now it's time to train our CNN model using the training data. Training involves feeding the training data to the model and adjusting its weights to minimize the loss function.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for `batch_size` and `epochs` in `model.fit()`.
2. **Choose appropriate values for `batch_size` and `epochs`.**
3. **Run the code cell to start training.** Observe the training progress, especially the loss and accuracy on both the training and validation sets.
4. **Experiment!** Change the `batch_size` and `epochs` and see how it affects the training process and the final performance.

```
1 # Cell 5: Model Training
2 # Students: Adjust the batch size and number of epochs. What happens if you change them?
3 model.fit(x_train, y_train, batch_size=128, epochs=15, validation_split=0.1) #Students: Fill in the blanks
```

Epoch 1/15

422/422 52s 121ms/step - accuracy: 0.9125 - loss: 0.2971 - val_accuracy: 0.9807 - val_loss: 0.0723

Epoch 2/15

422/422 50s 119ms/step - accuracy: 0.9724 - loss: 0.0913 - val_accuracy: 0.9843 - val_loss: 0.0554

Epoch 3/15

422/422 84s 125ms/step - accuracy: 0.9777 - loss: 0.0701 - val_accuracy: 0.9870 - val_loss: 0.0487

Epoch 4/15

422/422 81s 122ms/step - accuracy: 0.9826 - loss: 0.0572 - val_accuracy: 0.9887 - val_loss: 0.0401

Epoch 5/15

422/422 82s 122ms/step - accuracy: 0.9835 - loss: 0.0506 - val_accuracy: 0.9900 - val_loss: 0.0356

Epoch 6/15

422/422 80s 118ms/step - accuracy: 0.9864 - loss: 0.0445 - val_accuracy: 0.9902 - val_loss: 0.0369

Epoch 7/15

422/422 52s 124ms/step - accuracy: 0.9877 - loss: 0.0388 - val_accuracy: 0.9910 - val_loss: 0.0370

Epoch 8/15

422/422 81s 122ms/step - accuracy: 0.9884 - loss: 0.0378 - val_accuracy: 0.9907 - val_loss: 0.0368

Epoch 9/15

422/422 83s 125ms/step - accuracy: 0.9888 - loss: 0.0349 - val_accuracy: 0.9915 - val_loss: 0.0341

Epoch 10/15

422/422 81s 123ms/step - accuracy: 0.9899 - loss: 0.0319 - val_accuracy: 0.9910 - val_loss: 0.0330

Epoch 11/15

422/422 51s 122ms/step - accuracy: 0.9904 - loss: 0.0292 - val_accuracy: 0.9913 - val_loss: 0.0325

Epoch 12/15

422/422 83s 124ms/step - accuracy: 0.9918 - loss: 0.0267 - val_accuracy: 0.9922 - val_loss: 0.0329

Epoch 13/15

422/422 52s 123ms/step - accuracy: 0.9917 - loss: 0.0255 - val_accuracy: 0.9907 - val_loss: 0.0347

Epoch 14/15

422/422 52s 123ms/step - accuracy: 0.9917 - loss: 0.0262 - val_accuracy: 0.9917 - val_loss: 0.0308

Epoch 15/15

422/422 51s 121ms/step - accuracy: 0.9925 - loss: 0.0225 - val_accuracy: 0.9912 - val_loss: 0.0360

<keras.src.callbacks.history.History at 0x79240ef24c20>

Initial Training Results:

Using 128 filters in the second Conv2D layer, a batch size of 128, and 15 epochs, the model reached 99.25% training accuracy and 99.12% validation accuracy. The highest validation accuracy was approximately 99.22% at epoch 12. The model showed only a small difference between training and validation accuracy.

```
1 experiment_model = keras.Sequential(
2     [
3         keras.Input(shape=(28, 28, 1)),
```

```

4     layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
5     layers.MaxPooling2D(pool_size=(2, 2)),
6     layers.Conv2D(128, kernel_size=(3, 3), activation="relu"),
7     layers.MaxPooling2D(pool_size=(2, 2)),
8     layers.Flatten(),
9     layers.Dropout(0.5),
10    layers.Dense(num_classes, activation="softmax"),
11    ]
12 )
13
14 experiment_model.compile(
15     loss="categorical_crossentropy",
16     optimizer="adam",
17     metrics=["accuracy"]
18 )
19
20 experiment_history = experiment_model.fit(
21     x_train,
22     y_train,
23     batch_size=64,
24     epochs=10,
25     validation_split=0.1
26 )

```

```

Epoch 1/10
844/844 ————— 67s 77ms/step - accuracy: 0.9296 - loss: 0.2305 - val_accuracy: 0.9857 - val_loss: 0.0559
Epoch 2/10
844/844 ————— 81s 76ms/step - accuracy: 0.9760 - loss: 0.0779 - val_accuracy: 0.9883 - val_loss: 0.0451
Epoch 3/10
844/844 ————— 83s 77ms/step - accuracy: 0.9822 - loss: 0.0582 - val_accuracy: 0.9888 - val_loss: 0.0407
Epoch 4/10
844/844 ————— 81s 77ms/step - accuracy: 0.9847 - loss: 0.0492 - val_accuracy: 0.9913 - val_loss: 0.0352
Epoch 5/10
844/844 ————— 62s 73ms/step - accuracy: 0.9858 - loss: 0.0439 - val_accuracy: 0.9920 - val_loss: 0.0336
Epoch 6/10
844/844 ————— 87s 80ms/step - accuracy: 0.9877 - loss: 0.0388 - val_accuracy: 0.9922 - val_loss: 0.0288
Epoch 7/10
844/844 ————— 80s 77ms/step - accuracy: 0.9884 - loss: 0.0358 - val_accuracy: 0.9918 - val_loss: 0.0293
Epoch 8/10
844/844 ————— 63s 75ms/step - accuracy: 0.9898 - loss: 0.0315 - val_accuracy: 0.9908 - val_loss: 0.0302
Epoch 9/10
844/844 ————— 64s 76ms/step - accuracy: 0.9908 - loss: 0.0286 - val_accuracy: 0.9918 - val_loss: 0.0308
Epoch 10/10
844/844 ————— 64s 76ms/step - accuracy: 0.9906 - loss: 0.0273 - val_accuracy: 0.9917 - val_loss: 0.0312

```

Training Parameter Experiment:

I changed the batch size from 128 to 64 and reduced the number of epochs from 15 to 10 while keeping the CNN architecture the same. The experimental model reached 99.10% training accuracy and 99.13% validation accuracy at epoch 10, with its highest validation accuracy of 99.30% at epoch 9.

Compared with the initial run, which used a batch size of 128 and 15 epochs, the final validation accuracy was nearly identical. The smaller batch size caused the model to update its weights more frequently, while using fewer epochs reduced the overall training time. In this case, additional epochs did not provide a meaningful improvement in validation performance, suggesting that 10 epochs were sufficient for this model.

Explanation of Training Parameters:

- **batch_size**: This determines the number of training samples processed in each mini-batch during training. A larger batch size can speed up training but might require more memory. A smaller batch size can lead to more noisy updates but might generalize better.
- **epochs**: One epoch represents one complete pass through the entire training dataset. More epochs can potentially lead to better training but also increase the risk of overfitting, where the model learns the training data too well and performs poorly on unseen data.
- **validation_split=0.1**: This reserves 10% of the training data as a validation set. During training, the model's performance is evaluated on this validation set after each epoch. This helps to monitor for overfitting and tune hyperparameters.

Section 6: Model Evaluation - Assessing Performance on Test Data

After training, we need to evaluate our model's performance on the test dataset. This gives us an estimate of how well the model generalizes to unseen data.

Instructions:

1. Run the code cell below.

2. **Observe the output.** It will print the test loss and test accuracy.
3. **Think about the results.** Is the test accuracy satisfactory? How does it compare to the training and validation accuracy you observed during training?

```
1 # Cell 6: Model Evaluation
2 loss, accuracy = experiment_model.evaluate(x_test, y_test, verbose=0)
3 print(f"Test loss: {loss:.4f}")
4 print(f"Test accuracy: {accuracy:.4f}")
```

```
Test loss: 0.0253
Test accuracy: 0.9919
```

Model Evaluation:

The experimental model achieved 99.19% accuracy on the test dataset with a test loss of 0.0253. This result is very close to the training and validation accuracy, which suggests that the model generalized well to unseen data and did not show significant overfitting.

Section 7: Reflection and Answers to Questions

This is an important section! Take some time to reflect on what you have learned and answer the following questions in detail. Your thoughtful answers will demonstrate your understanding of the concepts covered in this assignment.

Reflection Questions:

1. **Conv2D Layer:** What is the role of the Conv2D layer? How do the `kernel_size` and the number of filters affect the learning process? *Hint: Experiment by changing these values in Cell 3.*

The Conv2D layer detects visual patterns in an image, such as edges, curves, and shapes. The `kernel_size` controls the size of the filter that moves across the image. A 3×3 kernel looks at small local areas at a time. The number of filters determines how many different features the model can learn. In my model, I increased the second Conv2D layer from 64 to 128 filters so the network could learn a larger variety of features, although this also increased the amount of computation required.

2. **MaxPooling2D Layer:** What is the purpose of the MaxPooling2D layer? How does it contribute to the model's performance? *Hint: Try removing or adding a MaxPooling2D layer and see what happens.*

The MaxPooling2D layer reduces the spatial size of the feature maps while keeping the strongest detected features. With a 2×2 pool, the model keeps the maximum value from each small region. This reduces the amount of data the network must process, lowers computational cost, and can help reduce overfitting while still preserving important visual information.

3. **One-Hot Encoding:** Why do we use one-hot encoding for the labels?

One-hot encoding converts each digit label into a vector representing one of the 10 possible classes. For example, the digit 3 is represented with a 1 in the position for class 3 and 0s everywhere else. This format works well with the softmax output layer and categorical cross-entropy loss because the model predicts a probability for each class.

4. **Flatten Layer:** Why do we need the Flatten layer before the Dense layer?

The Flatten layer converts the two-dimensional feature maps produced by the convolution and pooling layers into a one-dimensional vector. This is necessary because the Dense output layer expects its input in vector form.

5. **Optimizer and Loss Function:** What optimizer and loss function did you choose in Cell 4? Explain your choices. Why is categorical cross-entropy a suitable loss function for this task? Why is Adam a good choice of optimiser?

I used the Adam optimizer and categorical cross-entropy loss. Categorical cross-entropy is appropriate because this is a multi-class classification problem with 10 one-hot encoded classes. Adam is a good choice because it efficiently updates the model's weights and adapts the learning rate during training, which often allows the model to learn well without extensive manual tuning.

6. **Batch Size and Epochs:** How did you choose the batch size and number of epochs in Cell 5? What are the effects of changing these parameters? *Hint: Experiment!*

My initial training used a batch size of 128 and 15 epochs. I then experimented with a batch size of 64 and 10 epochs while keeping the model architecture the same. The experimental run reached 99.10% training accuracy and 99.13% validation accuracy, with a highest validation accuracy of 99.30%. This was very close to the original run, which suggests that the additional epochs were not necessary for this model. The smaller batch size also caused the model to update its weights more frequently.

7. **Dropout:** Why is the Dropout layer included in the model?

The Dropout layer helps reduce overfitting by randomly turning off a percentage of neurons during training. In this model, the dropout rate was 0.5, meaning about half of the units were temporarily ignored during each training update. This encourages the network to learn more general patterns instead of relying too heavily on particular neurons.

8. **Model Architecture:** Describe the overall architecture of your CNN. How many convolutional layers did you use? How many max pooling layers? What is the final dense layer doing?

My CNN used two convolutional layers and two MaxPooling2D layers. The first Conv2D layer used 32 filters, and I changed the second layer to 128 filters. After the convolution and pooling layers, the Flatten layer converted the feature maps into a vector, followed by a Dropout layer for regularization. The final Dense layer had 10 outputs with softmax activation, producing a probability for each digit from 0 through 9.

9. **Performance:** What accuracy did you achieve on the test set? Are you happy with the result? Why or why not? If you're not happy, what could you try to improve the performance?

The final experimental model achieved 99.19% test accuracy with a test loss of 0.0253. I am satisfied with this result because the test accuracy was very close to the training and validation accuracy, which suggests the model generalized well to unseen images. If I wanted to explore further improvements, I could test different numbers of filters, kernel sizes, dropout rates, optimizers, or additional convolutional layers.

Tips and Explanations:

- **Normalization:** Dividing the pixel values by 255 normalizes them to the range [0, 1]. This is important for training neural networks.
- **Reshaping:** The `reshape` operation adds a channel dimension to the images. For grayscale images, the channel dimension is 1.
- **One-Hot Encoding:** `to_categorical` converts the class labels (0-9) into one-hot encoded vectors.
- **Conv2D Parameters:** The `kernel_size` determines the size of the convolutional filter (e.g., 3x3). The number of filters determines how many different features are learned.
- **MaxPooling2D Parameters:** The `pool_size` determines the size of the pooling window (e.g., 2x2).
- **Optimizer:** The optimizer is the algorithm used to update the model's weights during training.
- **Loss Function:** The loss function measures the error between the model's predictions and the true labels.
- **Batch Size:** The batch size is the number of samples processed in each training iteration.
- **Epochs:** An epoch is one complete pass through the entire training dataset.
- **Dropout:** Dropout is a regularization technique that helps prevent overfitting.

Remember to run each cell to see its output. Experiment with the code and try to understand how different parameters affect the model's performance. Good luck! """"

Conclusion and Submission

Congratulations on completing this notebook assignment! You have successfully built and trained a Convolutional Neural Network to classify handwritten digits from the MNIST dataset. You've explored key concepts like convolutional layers, pooling layers, activation functions, optimizers, loss functions, and training procedures. To further solidify your understanding, consider the following:

- **Review your notebook:** Go back through each section, reread the explanations, and make sure you understand the code and the concepts.
- **Experiment further:** Try different CNN architectures, add more layers, change hyperparameters, and see how it affects the performance. Explore other optimizers or loss functions.
- **Reflect on your learning:** Think about the challenges you faced and how you overcame them. What were the most important takeaways for you from this assignment?

Submission Instructions

To submit your assignment:

1. **Save your notebook:** Ensure all your work, including code cells, outputs, and answers to reflection questions, is saved in the notebook.
2. **Print the notebook as a `.pdf` file** and submit it to Canvas.

Deadline: February, 12th

Double-click (or enter) to edit

Building a CNN for MNIST Handwritten Digit Classification

✓ Introduction

Welcome! In this assignment, you will build a Convolutional Neural Network (CNN) to classify handwritten digits from the famous MNIST dataset. This dataset is a classic in the field of computer vision and provides a great starting point for understanding image classification with deep learning.

This notebook is structured to guide you step-by-step through the process. You will load the data, preprocess it, define a CNN model, train it, and evaluate its performance. Throughout the assignment, you will have opportunities to experiment and deepen your understanding of the concepts.

Remember to:

- **Read all instructions carefully.**
- **Execute the code cells in order.**
- **Fill in the missing code sections marked as "Students: Fill in the blanks".**
- **Answer the reflection questions in the designated Markdown cell.**
- **Experiment and explore!** Change parameters, layers, and observe the effects.

Let's get started and build our MNIST digit classifier!

Section 1: Setting Up - Imports

Before we dive into building our CNN, we need to import the necessary libraries. These libraries provide pre-built tools and functions that will make our work much easier.

Instructions:

1. **Carefully review the code cell below.** It imports libraries from TensorFlow and Keras, which are powerful frameworks for building and training neural networks.
2. **Execute the code cell by selecting it and pressing [Shift + Enter] (or the "Run" button).**
3. **Ensure there are no error messages after running the cell.** If you encounter errors, double-check that you have TensorFlow and Keras installed in your environment.

```
# Cell 1: Imports
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
```

Explanation of Imports:

- **tensorflow as tf and keras**: TensorFlow is the main deep learning framework, and Keras is its high-level API that simplifies building and training models. We import TensorFlow as **tf** and Keras directly for easy access to their functionalities.
- **from tensorflow.keras import layers**: This imports the **layers** module from Keras, which provides various layers for building neural networks (like convolutional layers, dense layers, etc.).
- **from tensorflow.keras.datasets import mnist**: This imports the MNIST dataset directly from Keras datasets. This is very convenient for loading and using the MNIST data.
- **from tensorflow.keras.utils import to_categorical**: This imports the **to_categorical** function, which we will use to perform one-hot encoding of our labels.

✓ Section 2: Data Loading and Preprocessing

In this section, we will load the MNIST dataset and prepare it for training our CNN model. Preprocessing steps are crucial to ensure our data is in the right format for the model to learn effectively.

Instructions:

1. **Read through the code in the cell below.** Understand how it loads the MNIST dataset and what preprocessing steps are applied.
2. **Execute the code cell.**
3. **Examine the comments in the code** to understand each preprocessing step in detail.

```
# Cell 2: Data Loading and Preprocessing
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to be between 0 and 1
```

```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
# Add a channel dimension (for grayscale images, it's 1)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
```

```
# One-hot encode the labels
num_classes = 10
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-dataset/11490434/11490434>  0s 0us/step

Explanation of Data Preprocessing:

- **Loading the MNIST dataset:** `mnist.load_data()` loads the MNIST dataset, which is already split into training and testing sets (`(x_train, y_train)`, `(x_test, y_test)`). `x_train` and `x_test` contain the images (pixel data), and `y_train` and `y_test` contain the corresponding labels (digits 0-9).
- **Normalization:** `x_train = x_train.astype("float32") / 255.0` and `x_test = x_test.astype("float32") / 255.0` normalize the pixel values. Pixel values in images are typically in the range 0-255. Dividing by 255 scales them to the range 0-1. This normalization helps the neural network train faster and more effectively.
- **Adding Channel Dimension:** `x_train = x_train.reshape(-1, 28, 28, 1)` and `x_test = x_test.reshape(-1, 28, 28, 1)` reshape the data to add a channel dimension. Even though MNIST images are grayscale (single channel), CNNs in Keras expect input data to have a channel dimension. We reshape from `(number_of_images, 28, 28)` to `(number_of_images, 28, 28, 1)`. The `-1` in `reshape` means "infer the dimension based on the size of the array."
- **One-Hot Encoding:** `y_train = to_categorical(y_train, num_classes)` and `y_test = to_categorical(y_test, num_classes)` perform one-hot encoding on the labels. Instead of representing the digit '3' as a single number, one-hot encoding converts it into a vector `[0, 0, 0, 1, 0, 0, 0, 0, 0]`, where the 4th position (index 3) is 'hot' (value 1), and all other positions are 'cold' (value 0). This is a standard way to represent categorical labels for neural networks in multi-class classification problems. `num_classes = 10` specifies that we have 10 classes (digits 0-9).

✓ Section 3: Model Definition - Building the CNN

Now we will define the architecture of our Convolutional Neural Network (CNN). You will be building a sequential model using Keras layers.

Instructions:

1. **Carefully examine the code in the cell below.** Notice the structure of the `keras.Sequential` model.
2. **Fill in the missing parts** marked with `# Students: Fill in the blanks` to complete the model definition.
3. **Experiment!** You are encouraged to try different configurations for the layers, such as changing the number of filters in the convolutional layers, or adding more layers.

```
# Cell 3: Model Definition
# Build the CNN model. Students: Fill in the missing parts!
model = keras.Sequential(
    [
        keras.Input(shape=(28, 28, 1)), # Input layer
        layers.Conv2D(32, kernel_size=(3, 3), activation="relu"), # Convolutional layer 1
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 1
        # Students: Add another Conv2D layer here. Experiment with the number
        # layers.Conv2D(____, kernel_size=(____, ____), activation="____"), # Convolutional layer 2
        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"), # Convolutional layer 3
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 2
        # Students: Add another MaxPooling2D layer here if needed.
        # layers.MaxPooling2D(pool_size=(____, ____)), # Max pooling layer 3
        layers.Flatten(), # Flatten layer
        layers.Dropout(0.5), # Dropout layer
        layers.Dense(num_classes, activation="softmax"), # Output layer
    ]
)
```

Explanation of Layers:

- `keras.Input(shape=(28, 28, 1))`: This is the input layer of our model. It specifies the shape of the input images, which are 28x28 pixels with 1 channel (grayscale).
- `layers.Conv2D(32, kernel_size=(3, 3), activation="relu")`: This is a 2D Convolutional layer.
 - `32`: This is the number of filters (also called kernels). Each filter learns to detect specific features in the input image.
 - `kernel_size=(3, 3)`: This defines the size of the convolutional filter as 3x3 pixels.

- `activation="relu"`: ReLU (Rectified Linear Unit) is the activation function. It introduces non-linearity into the model, allowing it to learn complex patterns.
- `layers.MaxPooling2D(pool_size=(2, 2))`: This is a Max Pooling layer.
 - `pool_size=(2, 2)`: It reduces the spatial dimensions of the feature maps by taking the maximum value within each 2x2 window. This helps to reduce the number of parameters, control overfitting, and make the model more robust to small shifts and distortions in the input.
- `layers.Flatten()`: This layer flattens the 2D feature maps from the convolutional and pooling layers into a 1D vector. This is necessary to connect the convolutional part of the network to the fully connected (Dense) layers.
- `layers.Dropout(0.5)`: This is a Dropout layer.
 - `0.5`: This sets the dropout rate to 50%. During training, this layer randomly sets 50% of the input units to 0 at each update. This is a regularization technique that helps to prevent overfitting.
- `layers.Dense(num_classes, activation="softmax")`: This is the output Dense (fully connected) layer.
 - `num_classes`: This is set to 10 because we have 10 classes (digits 0-9).
 - `activation="softmax"`: Softmax activation ensures that the output values are probabilities, and they sum up to 1 across all classes. The output will be a vector of 10 probabilities, where each probability represents the model's confidence that the input image belongs to that specific digit class.

✓ Section 4: Model Compilation - Choosing Loss and Optimizer

Before we can train our model, we need to compile it. Compilation involves choosing an optimizer, a loss function, and metrics to evaluate the model's performance.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for the `loss` and `optimizer` parameters in `model.compile()`.
2. **Choose an appropriate loss function and optimizer** for this multi-class classification problem.
3. **In the Markdown cell after the code, explain your choices.** Why are these choices suitable for this task?

```
# Cell 4: Model Compilation
# Students: Choose an appropriate loss function and optimizer. Why did you choose categorical_crossentropy?
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
```

Explanation of Choices (To be filled by students in the reflection section):

- **Loss Function:** You need to choose a loss function that is appropriate for multi-class classification. Think about what kind of error we are trying to minimize when classifying digits into 10 categories.
- **Optimizer:** You need to choose an optimizer that will efficiently update the model's weights to minimize the loss function. Consider common optimizers used in deep learning.
- **Metrics:** We are using "accuracy" as a metric to evaluate the model's performance. Accuracy is a common metric for classification tasks, representing the percentage of correctly classified images.

✓ Section 5: Model Training - Fitting the Model to the Data

Now it's time to train our CNN model using the training data. Training involves feeding the training data to the model and adjusting its weights to minimize the loss function.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for `batch_size` and `epochs` in `model.fit()`.
2. **Choose appropriate values for `batch_size` and `epochs`.**
3. **Run the code cell to start training.** Observe the training progress, especially the loss and accuracy on both the training and validation sets.
4. **Experiment!** Change the `batch_size` and `epochs` and see how it affects the training process and the final performance.

```
# Cell 5: Model Training
# Students: Adjust the batch size and number of epochs. What happens if you change batch_size and epochs?
model.fit(x_train, y_train, batch_size=128, epochs=15, validation_split=0.1) #5
```

```
Epoch 1/15
422/422 ————— 42s 96ms/step - accuracy: 0.8836 - loss: 0.3781 - v
Epoch 2/15
422/422 ————— 41s 95ms/step - accuracy: 0.9653 - loss: 0.1156 - v
Epoch 3/15
422/422 ————— 41s 95ms/step - accuracy: 0.9729 - loss: 0.0892 - v
Epoch 4/15
422/422 ————— 40s 95ms/step - accuracy: 0.9772 - loss: 0.0739 - v
Epoch 5/15
```

```

422/422 ————— 40s 95ms/step - accuracy: 0.9805 - loss: 0.0653 - v
Epoch 6/15
422/422 ————— 40s 95ms/step - accuracy: 0.9818 - loss: 0.0585 - v
Epoch 7/15
422/422 ————— 41s 95ms/step - accuracy: 0.9836 - loss: 0.0520 - v
Epoch 8/15
422/422 ————— 40s 95ms/step - accuracy: 0.9846 - loss: 0.0495 - v
Epoch 9/15
422/422 ————— 40s 95ms/step - accuracy: 0.9847 - loss: 0.0478 - v
Epoch 10/15
422/422 ————— 40s 95ms/step - accuracy: 0.9872 - loss: 0.0419 - v
Epoch 11/15
422/422 ————— 40s 95ms/step - accuracy: 0.9870 - loss: 0.0406 - v
Epoch 12/15
422/422 ————— 41s 95ms/step - accuracy: 0.9866 - loss: 0.0402 - v
Epoch 13/15
422/422 ————— 40s 95ms/step - accuracy: 0.9880 - loss: 0.0372 - v
Epoch 14/15
422/422 ————— 40s 95ms/step - accuracy: 0.9881 - loss: 0.0364 - v
Epoch 15/15
422/422 ————— 41s 95ms/step - accuracy: 0.9889 - loss: 0.0334 - v
<keras.src.callbacks.history.History at 0x7d120e40d160>

```

Explanation of Training Parameters:

- **batch_size**: This determines the number of training samples processed in each mini-batch during training. A larger batch size can speed up training but might require more memory. A smaller batch size can lead to more noisy updates but might generalize better.
- **epochs**: One epoch represents one complete pass through the entire training dataset. More epochs can potentially lead to better training but also increase the risk of overfitting, where the model learns the training data too well and performs poorly on unseen data.
- **validation_split=0.1**: This reserves 10% of the training data as a validation set. During training, the model's performance is evaluated on this validation set after each epoch. This helps to monitor for overfitting and tune hyperparameters.

Section 6: Model Evaluation - Assessing Performance on Test Data

After training, we need to evaluate our model's performance on the test dataset. This gives us an estimate of how well the model generalizes to unseen data.

Instructions:

1. Run the code cell below.

2. **Observe the output.** It will print the test loss and test accuracy.
3. **Think about the results.** Is the test accuracy satisfactory? How does it compare to the training and validation accuracy you observed during training?

```
# Cell 6: Model Evaluation
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {loss:.4f}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Test loss: 0.0249
Test accuracy: 0.9908
```

✓ Section 7: Reflection and Answers to Questions

This is an important section! Take some time to reflect on what you have learned and answer the following questions in detail. Your thoughtful answers will demonstrate your understanding of the concepts covered in this assignment.

Reflection Questions:

1. **Conv2D Layer:** What is the role of the Conv2D layer? How do the `kernel_size` and the number of filters affect the learning process? *Hint: Experiment by changing these values in Cell 3.*
2. **MaxPooling2D Layer:** What is the purpose of the MaxPooling2D layer? How does it contribute to the model's performance? *Hint: Try removing or adding a MaxPooling2D layer and see what happens.*
3. **One-Hot Encoding:** Why do we use one-hot encoding for the labels?
4. **Flatten Layer:** Why do we need the Flatten layer before the Dense layer?
5. **Optimizer and Loss Function:** What optimizer and loss function did you choose in Cell 4? Explain your choices. Why is categorical cross-entropy a suitable loss function for this task? Why is Adam a good choice of optimiser?
6. **Batch Size and Epochs:** How did you choose the batch size and number of epochs in Cell 5? What are the effects of changing these parameters? *Hint: Experiment!*
7. **Dropout:** Why is the Dropout layer included in the model?
8. **Model Architecture:** Describe the overall architecture of your CNN. How many convolutional layers did you use? How many max pooling layers? What is the final dense layer doing?

9. **Performance:** What accuracy did you achieve on the test set? Are you happy with the result? Why or why not? If you're not happy, what could you try to improve the performance?

Tips and Explanations:

- **Normalization:** Dividing the pixel values by 255 normalizes them to the range [0, 1]. This is important for training neural networks.
- **Reshaping:** The `reshape` operation adds a channel dimension to the images. For grayscale images, the channel dimension is 1.
- **One-Hot Encoding:** `to_categorical` converts the class labels (0-9) into one-hot encoded vectors.
- **Conv2D Parameters:** The `kernel_size` determines the size of the convolutional filter (e.g., 3x3). The number of filters determines how many different features are learned.
- **MaxPooling2D Parameters:** The `pool_size` determines the size of the pooling window (e.g., 2x2).
- **Optimizer:** The optimizer is the algorithm used to update the model's weights during training.
- **Loss Function:** The loss function measures the error between the model's predictions and the true labels.
- **Batch Size:** The batch size is the number of samples processed in each training iteration.
- **Epochs:** An epoch is one complete pass through the entire training dataset.
- **Dropout:** Dropout is a regularization technique that helps prevent overfitting.

Remember to run each cell to see its output. Experiment with the code and try to understand how different parameters affect the model's performance. Good luck! """"

Answer the questions

1.- The Conv2D layer looks for lines, edges, curves, and different shapes in an image. The CNN learns these patterns during the training process using the images we provide.

The kernel is like a small window that moves across the image, looking for patterns. In this model, the kernel size is 3x3, so it examines small areas of the image at a time.

The model uses 32 filters to learn and detect different features from the image. Each filter can learn different patterns. Using more filters can allow the CNN to learn more

features, but it also requires more computation.

2.- The MaxPooling2D layer helps reduce the size of the information while keeping the important patterns identified by the Conv2D layer. With a pool size of 2x2, the layer looks at groups of four values and keeps the maximum value from each group. This helps reduce the amount of information that the network needs to process while keeping important features. It also makes the model more efficient.

3.- We use one-hot encoding to convert the label into a vector representation with 10 positions, one for each digit from 0 to 9. For example, if the image represents the number 3, the label 3 is converted into [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]. The 1 represents the correct class, and the other values are 0. This helps the model work with the 10 different classes and compare its prediction with the correct label.

4.- The Flatten layer takes the information from the Conv2D and MaxPooling layers and converts it into a one-dimensional vector. This is necessary because the Dense layer needs to receive the information as a vector. Flatten connects the convolutional part of the CNN with the Dense layer.

5.- In Cell 4, we chose categorical cross-entropy as the loss function because we have 10 classes and we use one-hot encoding. This loss function measures the error between the CNN prediction and the correct label. We chose Adam as the optimizer because it helps update the model's weights and reduce the error during the training process. We also use accuracy as a metric to see how many predictions the model classifies correctly.

6.- In Cell 5, we chose a batch size of 128 and 15 epochs. The batch size means that the model processes 128 images in each group before updating its weights. The 15 epochs mean that the model goes through the complete training dataset 15 times.

Increasing the number of epochs can help the model learn better and improve the accuracy, but too many epochs can cause overfitting. If we use fewer epochs, the training is faster, but the model may not learn enough. Changing the batch size changes how many images the model processes before updating its weights. A larger batch processes more images at one time, while a smaller batch makes updates more frequently.

7.- The Dropout layer is included to help prevent overfitting. In our model, we use 0.5, which means that during training, about 50% of the units are randomly turned off. This helps the model not depend too much on some neurons and helps it learn more general patterns.

8.- Our CNN model has one Conv2D layer with 32 filters and one MaxPooling2D layer. After that, we use a Flatten layer to convert the information into a vector, and a Dropout layer to help prevent overfitting. Finally, we have a Dense layer with 10 outputs, one for each digit from 0 to 9. All the information from the image passes through these layers and is processed by the CNN. At the end, the Dense layer identifies which number from 0 to 9 is the prediction.

9.- Our test accuracy was 99.08%, with a test loss of 0.0249. We are happy with this result because the model correctly classified most of the test images. The result shows that the CNN learned the patterns of the handwritten digits very well. However, we could still try to improve the model by changing the number of filters, adding another convolutional layer, or changing the number of epochs to see if the accuracy can improve.

Conclusion and Submission

Congratulations on completing this notebook assignment! You have successfully built and trained a Convolutional Neural Network to classify handwritten digits from the MNIST dataset. You've explored key concepts like convolutional layers, pooling layers, activation functions, optimizers, loss functions, and training procedures. To further solidify your understanding, consider the following:

- **Review your notebook:** Go back through each section, reread the explanations, and make sure you understand the code and the concepts.
- **Experiment further:** Try different CNN architectures, add more layers, change hyperparameters, and see how it affects the performance. Explore other optimizers or loss functions.
- **Reflect on your learning:** Think about the challenges you faced and how you overcame them. What were the most important takeaways for you from this assignment?

Submission Instructions

To submit your assignment:

1. **Save your notebook:** Ensure all your work, including code cells, outputs, and answers to reflection questions, is saved in the notebook.
2. **Print the notebook as a .pdf file** and submit it to Canvas.

Deadline: February, 12th

